

Comparable Interface

Natural Object Ordering

Shruthi Sree S
CSE [CYBER SECURITY]

Introduction

The **Comparable interface** in Java defines a *natural ordering* for objects, allowing automatic sorting. It is a key feature in the **java.lang package** that simplifies sorting operations by embedding comparison logic directly within classes.

Natural Ordering Concept

Natural ordering defines how objects of a class are sorted by default, such as numeric order or alphabetical sequence. Implementing **Comparable** allows Java to automatically sort objects using this intrinsic order, ensuring consistency and predictability in collections.

Usage in Collections and Arrays Sorting

The **Comparable interface** integrates seamlessly with key Java utilities like *TreeSet*, *TreeMap*, *Arrays.sort()*, and *Collections.sort()*, enabling automatic sorting without external comparators when objects implement a natural order.

Practical Example

```
1 import java.util.List;
2 import java.util.Arrays;
3 import java.util.Collections;
4
5 class Box implements Comparable<Box> {
6     int size;
7
8     Box(int size) {
9         this.size = size;
10    }
11
12    @Override
13    public int compareTo(Box other) {
14        return this.size - other.size;
15    }
16
17    public static void main(String[] args) {
18        List<Box> boxes = Arrays.asList(new Box(5), new Box(2), new Box(9));
19        Collections.sort(boxes);
20        for (Box b : boxes) {
21            System.out.println(b.size);
22        }
23    }
24 }
```

2

5

9

=== Code Execution Successful

Advantages and Benefits of Using Comparable

- 1) Using Comparable provides **built-in natural ordering**
- 2) Eliminates external comparators when one order suffices
- 3) Improves code maintenance by embedding sorting logic within classes.